

Exercise 8 (alternative): A Harness Roadmap for Your Team

Session: 8, Bringing It All Together

Duration: 20 minutes, plus a 10-minute plenary

Project: None. You think about one real team that you know well. You need a text file or a sheet of paper, nothing else.

Goal

1. You score one real team on the five layers of the harness pyramid.
2. You mark the lowest layer that is not yet in place. That layer is your lever: the place where one change helps your team most.
3. You write a five-line roadmap and one first step for back at work.

Before you start

Titles in *italics* are slide titles from this session.

Your notes. Write in a text file or on paper. Nobody collects them. At the end, you read two lines aloud.

Pick one team. Choose one real team with one codebase: the team you work in after this course. Not your whole company. If you have no next team yet, take the last team you worked in. Write the team's name at the top of your notes.

Work alone. Do not talk to your neighbours during the 20 minutes. In the plenary at the end, the whole room talks together.

Time. The trainer calls the plenary 20 minutes after the start. Stop then, even if your roadmap is not finished.

The pyramid. The slide *A Real Roadmap: One Team's Harness Pyramid* shows five layers. A team builds them from the bottom up. Each block from the slide *Beyond This Course: Six Building Blocks* sits on one layer:

1. **Ground rules:** how the team works with agents. Which files, which tools, who reviews. This layer is not a block on that slide. It is the team's own agreements, written down.
2. **Guides:** what steers the agent before it acts. The blocks context files (`CLAUDE.md` , `AGENTS.md`), specs and plans, MCP servers, and skills.

3. **Sensors**: what checks the agent after it acts. The block hooks, plus tests, quality gates and review.
4. **Distribution**: how the guides and sensors reach every teammate. The block packaging: a plugin or a marketplace, so one install command gives a new teammate everything.
5. **Proof**: how you show that a change to the harness helped. The block proof. A number you track, or a benchmark repo: fixed tasks that you run again after every change.

Your lever. The lowest layer that is not yet in place. Task 2 explains why one change there helps most.

Tasks

1. Score your team (6 min)

You produce five scores in your notes, one per layer.

1. For each layer, write one of three scores:
 - **in place**: most of the team uses or follows it every week.
 - **partly**: some people or some repos use it, but not most.
 - **missing**: nobody has it.
2. Next to each score, write one line: what exists, or what does not. Example: *"Guides: partly. One repo has a CLAUDE.md , the others have nothing."* Score each layer as a whole, with the question in the table.

These questions help you decide:

Layer	Question
Ground rules	Has the team agreed how it works with agents: which files, which tools, who reviews?
Guides	Does the agent find your build commands, conventions and specs before it starts?
Sensors	Does the agent hear about a failure before a person finds it?
Distribution	Does a new teammate get the same guides and sensors with one install?
Proof	Can you show that last month's harness change helped?

Done when: your notes hold five scores, each with one line on what exists.

2. Find your lever (4 min)

You mark one layer in your notes as your lever.

1. Start at the bottom of the pyramid, at ground rules.
2. Go up, one layer at a time. Stop at the first layer that is not **in place**. A score of **partly** also counts as not in place. Write "my lever" next to that layer. If all five layers are in place, take the layer with the weakest line from task 1.
3. Write one sentence: why does this layer help your team most?

Take the lowest layer, even if a higher layer feels like the bigger problem. Each layer needs the layers below it. For example, guides that the team never agreed on get ignored, and proof without guides and sensors has nothing to measure. Most teams score proof as missing, but proof is built last.

Done when: one layer is marked as your lever, with one sentence on why.

3. Write your roadmap (10 min)

You produce five roadmap lines, one per layer, and one first step.

1. Write one line per layer, from the bottom up. Each line says what your team adds next on that layer. A layer that is **in place** gets the words "keep as is" and nothing more.
2. Name a file, a repo, a tool, a person or a number in each line that adds something. "Better docs" names nothing. "An `AGENTS.md` in the payments repo" names a file and a repo.
3. Under your lever line, copy your sentence from task 2, with "Why:" in front. Then write your **first step**: the first thing you do back at work. It must fit in one morning, about three hours. For ground rules, a first step can be a one-page proposal for the next team meeting.

Example roadmap, in the shape you can copy:

Team: payments, 5 developers, Java

1. Ground rules (in place): keep as is.
2. Guides (missing, my lever): an AGENTS.md in the payments repo, with the build command, the folder layout and the two rules we always repeat in review.
Why: every session starts from zero, so every review repeats the same comments.
First step: write AGENTS.md for the payments repo, on my first morning back.
3. Sensors (partly): a PostToolUse hook that runs the linter after every edit.
4. Distribution (missing): a plugin in our team marketplace, so every teammate installs the same guides and sensors.
5. Proof (missing): count the PRs that get a second review round, for three months.

Done when: your notes hold five roadmap lines. Your lever line has a why sentence and a first step that fits in one morning.

Plenary (10 min)

The whole room talks together. The trainer leads three short rounds:

1. **Hands up.** The trainer names each layer. Raise your hand when your lever is named. The trainer writes the count per layer on the board.
2. **Read aloud.** The trainer asks four or five people to read their lever line and their first step. Each person reads only those two lines, not the why sentence.
3. **The room.** Which layer do most teams need first, and why? Then one last question: what do you still want to learn?

Have your lever line and first step ready to read.